

Mindmap React 组件化方案

从应用内功能到可安装、可组合、可二次开发的 React 组件包

技术设计与实施说明

本文记录当前 mindmap 的组件化封装思路、公共 API、状态边界、样式隔离和发布验证方案，面向需要在其他 React 项目中直接集成并继续开发的工程团队。

版本 0.1.0 · React 18/19 · React Flow · ELK · Zustand

1. 改造目标

当前 mindmap 已具备完整的树编辑能力，但最初是围绕单个 Next.js 页面构建的应用内功能。组件化改造的核心不是简单移动文件，而是重新定义组件边界、状态所有权和宿主项目的接入方式。

目标结果

- 提供可直接安装的 React 组件包，支持 ESM、CJS、TypeScript 类型声明和独立 CSS。
- 同时提供完整编辑器和低层组合接口，覆盖开箱即用与深度二次开发。
- 同一页面支持多个独立实例，实例之间不共享选择、历史、快捷键或持久化 key。
- 宿主可以选择完全控制数据，也可以使用内部状态，并按需启用 localStorage。
- 保留编辑、布局、拖拽、折叠、撤销重做、导入导出和配色能力。

典型接入

```
import { MindMapEditor } from "@your-scope/mindmap";  
import "@your-scope/mindmap/styles.css";
```

```
<MindMapEditor />
```

组件填充宿主容器，因此父元素必须具有明确的宽度和高度。

2. 分层架构

组件包按纯业务模型、实例状态、画布渲染和完整界面四层组织。每层只依赖更底层能力，避免把 Next.js 页面逻辑或浏览器存储逻辑渗透到模型层。

层次	职责	对外价值
Model	树结构、命令、校验、序列化、可见节点	可在服务端、测试或自定义 UI 中复用
Store	实例级选择、编辑、历史和变更通知	Provider 隔离状态，多实例安全
Canvas	React Flow、ELK、节点渲染、拖拽和快捷键	可嵌入的核心画布
Editor	标题栏、工具栏、配色、文件和帮助	开箱即用的完整体验

推荐的组合关系

```
MindMapEditor
  ■■ MindMapProvider
    ■■ ReactFlowProvider
      ■■ MindMapCanvas
        ■■ Topic renderer
        ■■ Edge renderer
        ■■ ELK layout
```

完整编辑器只是 Provider、Canvas 和外围 UI 的组合，不是唯一入口。宿主可以保留 Provider 和 Canvas，完全替换工具栏与业务操作。

3. 公共 API 设计

公共 API 以 React Flow 的使用方式为参考：组件负责渲染和交互，数据、类型和样式都从包入口显式导出。

完整编辑器

```
type MindMapEditorProps = {
  value?: MindMapTree;
  defaultValue?: MindMapTree;
  onChange?: (tree, detail) => void;
  persistence?: { documentKey: string; paletteKey?: string; debounceMs?: number };
  locale?: "zh-CN" | "en";
  messages?: Partial<MindMapMessages>;
  palettes?: MindMapPalette[];
};
```

低层组合接口

```
function CustomEditor() {
  return (
    <MindMapProvider defaultValue={tree} onChange={save}>
      <CustomToolbar />
      <div style={{ height: 600 }}>
        <MindMapCanvas locale="en" />
      </div>
    </MindMapProvider>
  );
}
```

useMindMap 返回树、选中状态、编辑状态、canUndo/canRedo，以及 select、edit、execute、addChild、addSibling、deleteSelected、undo、redo、fitView、zoomIn、zoomOut 等操作。

导出	用途
MindMapTree / MindMapNode	宿主保存、受控渲染和业务数据处理
MindMapCommand	统一表达新增、删除、移动、编辑、折叠和导入
parseDocument / serializeDocument	JSON 备份和外部数据校验
applyCommand / visible / descendants	无 UI 的树操作和派生数据计算

4. 数据与状态边界

组件化最关键的设计决定是明确数据谁负责。组件支持受控和非受控两种模式，但两种模式不能混用。

受控模式

```
const [tree, setTree] = useState(initialTree);
```

```
<MindMapEditor
  value={tree}
  onChange={(nextTree, detail) => {
    audit(detail);
    setTree(nextTree);
  }}
/>
```

- 宿主是树数据的唯一来源，适合协作、路由切换、服务端保存和审计。
- 组件内部仍维护选择、编辑态和撤销重做历史，但外部树替换会清理不再可靠的历史。

非受控模式

```
<MindMapEditor
  defaultValue={initialTree}
  onChange={(tree, detail) => backup(tree, detail)}
/>
```

- 组件内部管理树和历史，宿主只订阅变更。
- 默认不访问 localStorage，避免污染宿主项目和造成多个实例 key 冲突。

变更来源

source	含义
user	用户执行编辑、增删、移动、折叠或导入
undo	撤销业务命令产生的树变化
redo	重做业务命令产生的树变化
import	导入 JSON 文档替换当前树

5. 持久化、导入导出与国际化

持久化是宿主可选择的能力，不应成为组件不可见的副作用。通过 `documentKey` 和 `paletteKey` 显式隔离不同 `mindmap` 实例。

```
<MindMapEditor
  persistence={{
    documentKey: "project-a.mindmap",
    paletteKey: "project-a.palette",
    debounceMs: 300,
  }}
/>
```

- 首次挂载时读取并校验 `version 1` 文档；数据损坏时保留默认树并报告错误。
- 树变化采用防抖写入，页面隐藏时执行一次同步保存。
- 存储异常只影响持久化，不阻断继续编辑；宿主仍可使用 `onChange` 备份。

导入导出

完整编辑器提供 JSON 导入和导出按钮；低层 `Canvas` 不强制提供文件操作，宿主可以使用 `parseDocument`、`serializeDocument` 和 `execute({ type: "import" })` 自建流程。

国际化

内置 `zh-CN` 和 `en` 两套文案，覆盖按钮、提示、错误、快捷键说明、拖拽提示和无障碍标签。`messages` 采用 `Partial` 类型，允许只覆盖宿主关心的文案。

6. 样式与宿主环境隔离

原实现依赖 app/globals.css 中的 body、button 和全局通配符规则。组件包不能继续依赖这些应用级规则，否则会在宿主项目中产生不可预测的样式污染。

- 样式移动到 components/mindmap/styles.css，并通过独立入口 styles.css 发布。
- 所有编辑器规则限定在 .mindmap-root 作用域内。
- React Flow 必需 CSS 合并进组件 CSS，使用方无需额外寻找内部依赖。
- 编辑器根节点填充父容器，不再强制 100dvh；全屏高度由示例应用或宿主决定。
- CSS 变量保留为主题扩展点，例如 --map-root、--map-canvas 和 --branch。

宿主接入约束

项目	接入要求
Vite / CRA	安装包并导入 styles.css，父容器设置宽高
Next.js	编辑器放在 Client Component 中，样式可在 layout 导入
多实例	每个实例使用独立 Provider 和独立 persistence key

7. 构建、发布与验证

构建链使用 tsup 生成 ESM、CJS 和类型声明，使用 esbuild 打包 CSS。package.json 通过 exports 明确约束公共入口。

```
"exports": {
  ".": {
    "types": "./dist/index.d.ts",
    "import": "./dist/index.js",
    "require": "./dist/index.cjs"
  },
  "./styles.css": "./dist/style.css"
}
```

验证分层

- 模型和布局：树命令、导入校验、折叠、移动、历史和 ELK 非重叠布局。
- Provider：多实例隔离、受控同步、外部替换、持久化 key 和错误处理。
- 组件：编辑、中文输入法、键盘导航、拖拽、配色、导入导出和容器适配。
- 发布物：pnpm pack 后在独立临时项目中安装 tarball，验证 ESM、类型和 CSS exports。

当前验证结果

命令	结果
pnpm typecheck	通过
pnpm lint	通过
pnpm test	37 个测试通过
pnpm build	库、CSS 和 Next.js 构建通过
pnpm pack	生成 tarball 并完成消费验证

8. 二次开发建议

组件包将通用能力与产品决定分开。宿主项目可以在不修改核心画布的情况下替换产品层行为。

- 需要后端保存或协作时，优先使用受控模式，将 onChange 接入业务状态或同步层。
- 需要自定义命令时，使用 MindMapCommand 和 execute，避免直接修改 tree 对象。
- 需要自定义工具栏时，保留 MindMapProvider，使用 useMindMap 组合操作，并把 MindMapCanvas 放进有尺寸的容器。
- 需要完全替换视觉风格时，导入组件 CSS 后用根 className 和 CSS 变量扩展。
- 需要添加新的文案或品牌信息时，使用 locale、messages、title 和 palette 配置。

推荐的演进方向

方向	建议
发布	将 @your-scope/mindmap 替换为实际 npm scope，并接入 CI 发布流程
性能	节点规模扩大后，将 ELK 布局移入 Web Worker
扩展	增加导出适配器、主题 token 和可插拔节点渲染器

结论：组件化的核心是建立稳定的实例边界和公共契约。宿主通过 Provider、受控数据和显式 CSS 接入，就可以在保留现有编辑能力的同时获得可安装、可组合、可持续演进的 mindmap 基础设施。